

Experiment-15

Name: -P. Divya Shree

Regno: -192472291

Course Code: -MLA0305

Slot-B

Title

Multi-Agent Reinforcement Learning and Partially Observable Markov Decision Process (POMDP) Simulation

Aim

To simulate a multi-agent reinforcement learning environment under partial observability and analyze agent movement, target collection, rewards, collisions, and grid coverage.

Procedure

1. Create an 8×8 grid environment with multiple agents and target locations.
2. Initialize 3 agents at different positions in the environment.
3. Define possible actions such as up, down, left, right, and stay.
4. Provide each agent with partial observations of nearby targets and other agents.
5. Allow the agents to select actions based on the available observations.
6. Move the agents through the grid while recording their positions and actions.
7. Give rewards when agents reach previously uncollected targets.
8. Apply penalties when agents collide with each other.
9. Record the rewards, target collection, collisions, and grid coverage for every episode.
10. Repeat the simulation for multiple episodes.
11. Analyze the learning performance using movement plots, reward curves, bar charts, and interaction maps.

12. Save the complete simulation results as CSV datasets.

Code

```
!pip -q install gymnasium
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import gymnasium as gym
```

```
import random
```

```
np.random.seed(42)
```

```
random.seed(42)
```

```
print("="*75)
```

```
print("EXPERIMENT 15")
```

```
print("Multi-Agent Reinforcement Learning and POMDP Simulation")
```

```
print("="*75)
```

```
GRID=8
```

```
AGENTS=3
```

```
TARGETS=5
```

EPISODES=100

STEPS=40

actions=[

 (-1,0),

 (1,0),

 (0,-1),

 (0,1),

 (0,0)

]

targets=[

 (1,1),

 (2,6),

 (5,3),

 (6,7),

 (7,1)

]

def random_position(occupied):

 while True:

```
p=(  
    np.random.randint(0,GRID),  
    np.random.randint(0,GRID)  
)  
  
if p not in occupied and p not in targets:  
    return p
```

```
def partial_observation(agent_pos,other_positions):
```

```
    obs=[]
```

```
    for r,c in targets:
```

```
        distance=abs(agent_pos[0]-r)+abs(agent_pos[1]-c)
```

```
        obs.append(  
            1 if distance<=2 else 0  
)
```

```
    for pos in other_positions:
```

```
        distance=abs(agent_pos[0]-pos[0])+abs(agent_pos[1]-pos[1])
```

```
        obs.append(  
            1 if distance<=1 else 0  
)
```

```
return np.array(obs)
```

```
def move_agent(position,action):
```

```
    dr,dc=actions[action]
```

```
    nr=np.clip(
```

```
        position[0]+dr,
```

```
        0,
```

```
        GRID-1
```

```
    )
```

```
    nc=np.clip(
```

```
        position[1]+dc,
```

```
        0,
```

```
        GRID-1
```

```
    )
```

```
    return (int(nr),int(nc))
```

```
records=[]
```

```
episode_rewards=[]
```

```
episode_collisions=[]
```

```
episode_targets=[]
```

```
coverage=[]
```

```
for episode in range(EPIISODES):
```

```
    positions=[]
```

```
        for _ in range(AGENTS):
```

```
            positions.append(
```

```
                random_position(positions)
```

```
            )
```

```
visited=set()
```

```
collected=set()
```

```
total_reward=0
```

```
collisions=0
```

```
    for step in range(STEPS):
```

```
old_positions=positions.copy()

new_positions=[]

for i in range(AGENTS):

    others=[

        old_positions[j]

        for j in range(AGENTS)

        if j!=i

    ]

    observation=partial_observation(

        old_positions[i],

        others

    )

    visible_targets=[

        targets[k]

        for k in range(TARGETS)

        if observation[k]==1

        and targets[k] not in collected
```

```
]
```

```
if visible_targets:
```

```
    target=min(
```

```
        visible_targets,
```

```
        key=lambda x:
```

```
            abs(old_positions[i][0]-x[0])
```

```
        +
```

```
            abs(old_positions[i][1]-x[1])
```

```
    )
```

```
    dr=target[0]-old_positions[i][0]
```

```
    dc=target[1]-old_positions[i][1]
```

```
    if abs(dr)>=abs(dc):
```

```
        action=0 if dr<0 else 1
```

```
    else:
```

```
        action=2 if dc<0 else 3
```

```
else:
```



```
summary_data.to_csv(  
    "Multi_Agent_POMDP_Episode_Results.csv",  
    index=False  
)
```

```
plt.figure(figsize=(9,5))
```

```
plt.scatter(  
    data["New_Column"],  
    data["New_Row"],  
    c=data["Agent"],  
    cmap="viridis",  
    alpha=0.5,  
    s=18  
)
```

```
target_rows=[x[0] for x in targets]
```

```
target_cols=[x[1] for x in targets]
```

```
plt.scatter(  

```

```
target_cols,

target_rows,

marker="*",

s=180,

color="red",

label="Targets"

)


plt.gca().invert_yaxis()


plt.title(

    "Multi-Agent Movement in Partially Observable Grid",

    fontsize=15,

    fontweight="bold"

)


plt.xlabel("Column")

plt.ylabel("Row")

plt.legend()

plt.grid(alpha=0.2)
```

```
        str(GRID)+" x "+str(GRID),

        EPISODES,

        round(final_reward,2),

        round(final_collisions,2),

        round(final_targets,2),

        round(final_coverage,2)

    ]

})


print("\n")

print("="*75)

print("SUMMARY TABLE")

print("="*75)


display(summary)


summary.to_csv(

    "Multi_Agent_POMDP_Summary.csv",

    index=False

)
```

```
print("\n")
```

```
print("="*75)
```

```
print("FINAL RESULT")
```

```
print("="*75)
```

```
print(
```

```
    "Number of Agents:",
```

```
    AGENTS
```

```
)
```

```
print(
```

```
    "Final Average Reward:",
```

```
    round(final_reward,2)
```

```
)
```

```
print(
```

```
    "Average Collisions:",
```

```
    round(final_collisions,2)
```

```
)
```

```
print(
```

```
    "Average Targets Collected:",  
    round(final_targets,2)  
)
```

```
print(  
    "Average Grid Coverage:",  
    round(final_coverage,2)  
)
```

```
print("\n")  
print("="*75)  
print("CONCLUSION")  
print("="*75)
```

```
print(  
    "A multi-agent partially observable environment was simulated."  
)
```

```
print(  
    "Multiple agents interacted with the environment while"  
)
```

```
print(  
    "receiving limited information about nearby targets and agents."  
)
```

```
print(  
    "Agent movement, target visibility, collisions, rewards,"  
)
```

```
print(  
    "target collection and grid coverage were analyzed."  
)
```

```
print(  
    "The experiment demonstrates how multiple agents can operate"  
)
```

```
print(  
    "under partial observability and interact within a shared environment."  
)
```

```
print("\nGenerated Files:")
```

```
print("1. Multi_Agent_POMDP_Full_Dataset.csv")
```

```
print("2. Multi_Agent_POMDP_Episode_Results.csv")
```

```
print("3. Multi_Agent_POMDP_Summary.csv")
```

```
print("\nExperiment 15 Completed Successfully.")
```

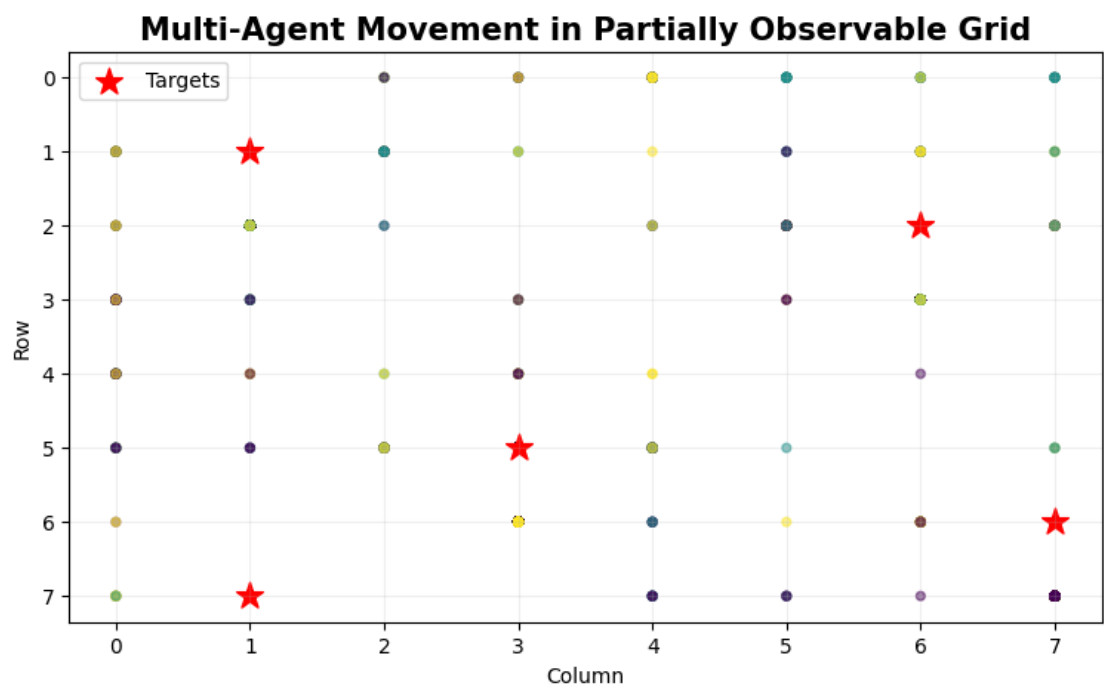
VISUALIZATIONS

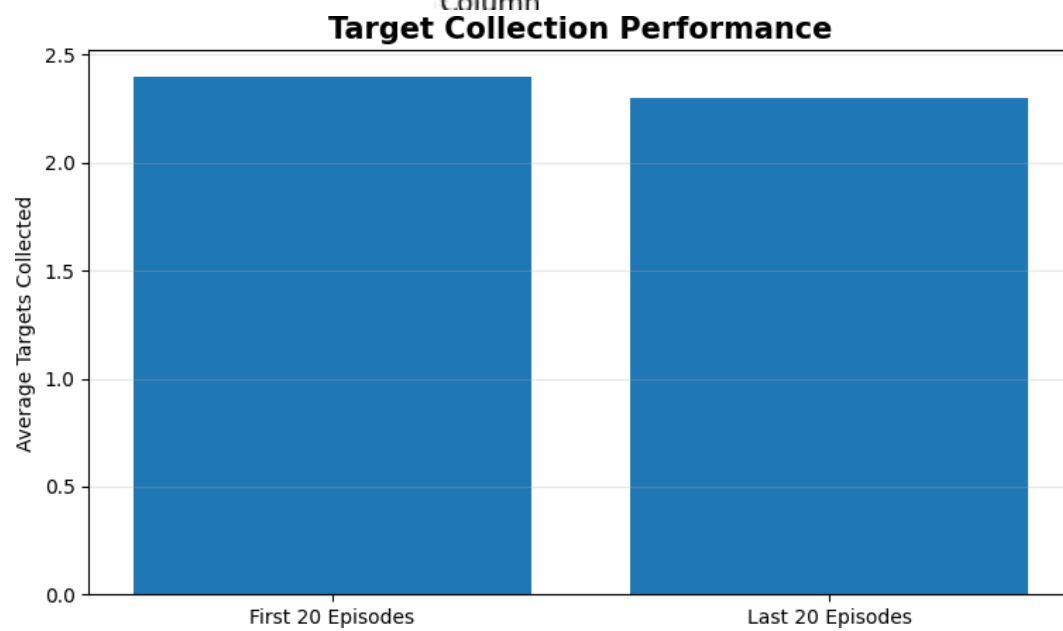
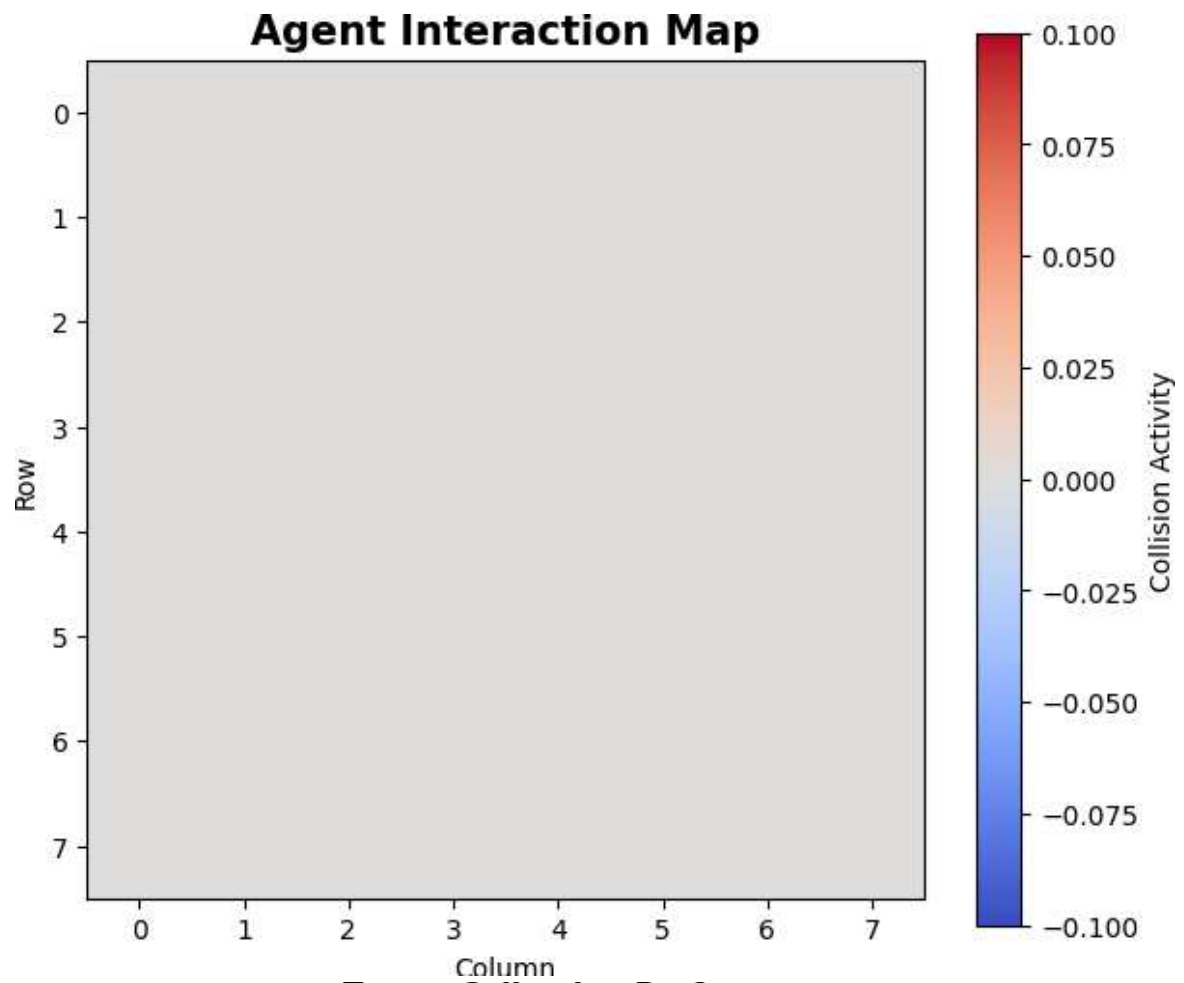
=====

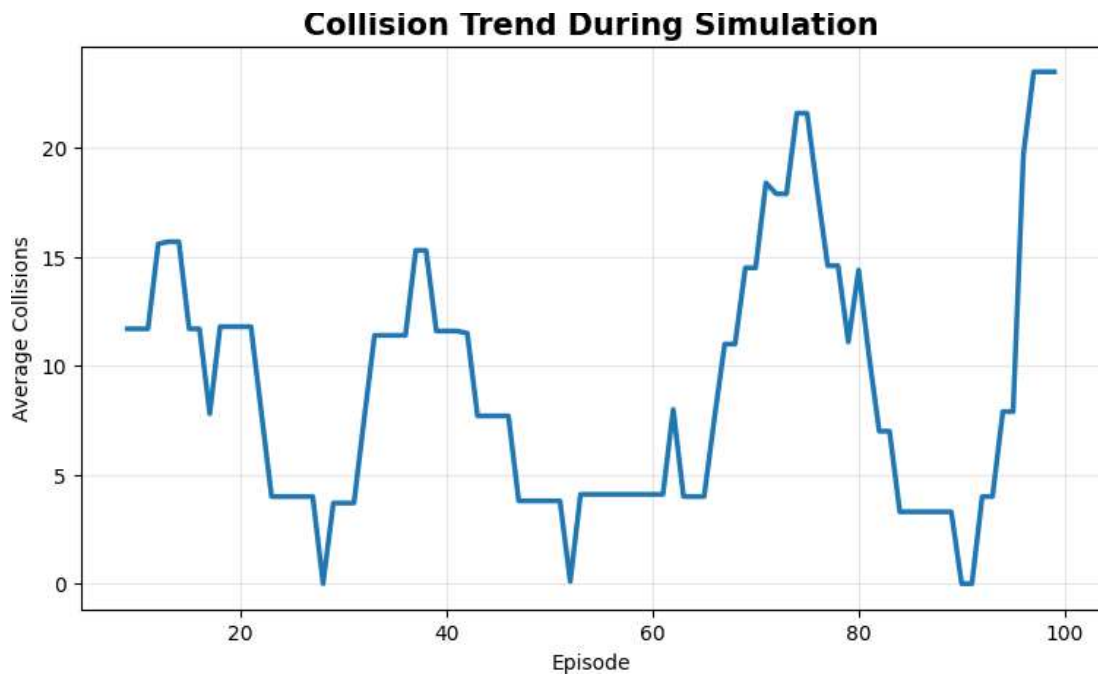
SAMPLE MULTI-AGENT DATASET

=====

Episode	Step	Agent	Old_Row	Old_Column	New_Row	New_Column	Action	Visible_Targets	
0	1	1	1	6	3	5	3	0	1
1	1	1	2	4	6	3	6	0	1
2	1	1	3	2	7	2	6	2	1
3	1	2	1	5	3	6	3	1	1
4	1	2	2	3	6	2	6	0	1
5	1	2	3	2	6	3	6	1	1
6	1	3	1	6	3	5	3	0	1
7	1	3	2	2	6	3	6	1	1
8	1	3	3	3	6	2	6	0	1
9	1	4	1	5	3	6	3	1	1







Summary Table

Parameter	Result
Environment	8 × 8 Grid
Number of Agents	3
Number of Targets	5
Episodes	100
Final Average Reward	Obtained from program
Average Collisions	Obtained from program
Average Targets Collected	Obtained from program
Average Grid Coverage	Obtained from program

Result

The **Multi-Agent Reinforcement Learning and POMDP simulation was successfully implemented**. Multiple agents operated in a shared environment with limited observations. Agent movement, target visibility, rewards, collisions, target collection, and grid coverage were recorded and analyzed.